



LESSON 10: FUNCTIONS

Reusable Code Modules

Module: Code Organization & Abstraction

Neon City Cyber Academy

Junior Agent Training Protocol

Your Team Today

Emily Chen - Handler

"Functions are how professionals organize code. Write once, use everywhere. This is the DRY principle: Don't Repeat Yourself."

Cole Martinez - Tech Lead

"When you call a function, the CPU pushes a new stack frame. Understanding this makes you a real programmer, not just a code writer."



Quick Review: Code We've Written

So far, all our code runs **top to bottom, once.**

Problem: What if we need the same code multiple times?

```
# Bad: Repetitive code
print("Starting system...")
print("Loading modules...")
print("System ready!")

# ... later ...
print("Starting system...")
print("Loading modules...")
print("System ready!")
```

The Solution: Functions

Function: A named block of reusable code.

```
def boot_system():  
    print("Starting system...")  
    print("Loading modules...")  
    print("System ready!")  
  
# Use it!  
boot_system() # Call once  
boot_system() # Call again!
```

Result: Write once, use anywhere!



Function Anatomy

```
def function_name():  
    # Code block (indented)  
    print("This runs when called")
```

Parts:

1. `def` - keyword (define)
2. `function_name` - your chosen name
3. `()` - parentheses (parameters go here later)
4. `:` - colon
5. **Indented block** - the function body



Emily Explains: Why Functions?

"Three big reasons:

1. **DRY** - Don't Repeat Yourself
2. **Organization** - Break big problems into small pieces
3. **Abstraction** - Hide complexity behind a simple name

Instead of 50 lines of code, you can write:

```
process_data()
```

Much cleaner!"



Example: Greeting Function

```
def greet():  
    print("👋 Hello, Agent!")  
    print("Welcome to Neon City")  
    print("Ready for your mission?")  
  
# Call it  
greet()
```

Output:

```
👋 Hello, Agent!  
Welcome to Neon City  
Ready for your mission?
```



Calling Functions Multiple Times

```
def countdown():  
    print("3...")  
    print("2...")  
    print("1...")  
    print("🚀 Launch!")  
  
countdown() # First launch  
print("---")  
countdown() # Second launch
```

Each call runs the entire function!



Cole's Under The Hood: Stack Frames

"When you call a function:

1. CPU creates a **stack frame** in memory
2. Jumps to the function code
3. Executes line by line
4. Returns to where it was called from
5. Destroys the stack frame

The **call stack** is how your computer tracks function calls.
This is crucial for recursion (later module)!"



Functions Can Call Other Functions!

```
def print_header():  
    print("=" * 30)  
    print("  NEON CITY CYBER ACADEMY")  
    print("=" * 30)  
  
def print_footer():  
    print("=" * 30)  
  
def display_welcome():  
    print_header()  
    print("Welcome, Agent!")  
    print_footer()  
  
display_welcome()
```



Challenge #1: Create a Separator Function

Mission: Write a function called `separator` that prints a line of 40 dashes.

```
# Your code here

# Test it
print("Section 1")
separator()
print("Section 2")
separator()
```

Expected Output:

```
Section 1
-----
Section 2
-----
```



Solution: Separator Function

```
def separator():  
    print("-" * 40)  
  
# Test it  
print("Section 1")  
separator()  
print("Section 2")  
separator()
```

Simple, but powerful! Reusable code at its finest.



Function Naming Conventions

Python rules:

- Use **lowercase** with **underscores**
- Must start with letter or underscore
- Can't be a Python keyword

```
# Good names
def calculate_total():
def print_report():
def send_alert():

# Bad names
def CalculateTotal(): # Wrong style (not PEP 8)
def 2calculate():      # Starts with number
def print():           # Overwrites built-in
```



Emily's Naming Wisdom

"Function names should be **verbs** or **verb phrases**:



`calculate_tax()`



`send_message()`



`validate_input()`



`format_output()`

NOT:



`tax()` - unclear what it does



`data()` - is this getting or setting?

Clear names = maintainable code!"



The Function Definition vs Call

Definition: Creates the function (doesn't run it)

```
def say_hello():  
    print("Hello!")  
# Nothing printed yet!
```

Call: Runs the function

```
say_hello() # NOW it prints!
```

Common beginner mistake: Defining without calling!



Common Error: Forgetting Parentheses

```
def greet():  
    print("Hi!")
```

`greet` # ❌ No output! Missing ()

Without `()`, you're referencing the function, not calling it!

Fix:

`greet()` # ✅ Prints "Hi!"



Common Error: Indentation

```
def my_function():  
print("Wrong!") # ✗ IndentationError
```

Fix:

```
def my_function():  
    print("Right!") # ✓ Indented
```

Function body MUST be indented!



Example: ASCII Art Function

```
def print_robot():  
    print("  🤖  ")  
    print(" /|\ \"")  
    print(" /  \ \"")  
  
def print_rocket():  
    print("  🚀  ")  
    print(" /|\ \"")  
    print(" / | \ \"")  
  
print_robot()  
print()  
print_rocket()
```



Cole's Organization Tip

"As your programs grow:

- Put related functions together
- Add blank lines between functions (PEP 8: 2 blank lines)
- Comment complex functions

Large programs might have **hundreds** of functions.

Organization matters!"

Challenge #2: Drawing Functions

Mission: Create three functions:

- `draw_square()` - prints a 3x3 square of `*`
- `draw_line()` - prints a horizontal line of 10 `-`
- `draw_art()` - calls both functions to make a design

```
# Your code here
```



Solution: Drawing Functions

```
def draw_square():  
    for i in range(3):  
        print("* * *")
```

```
def draw_line():  
    print("-" * 10)
```

```
def draw_art():  
    draw_line()  
    draw_square()  
    draw_line()
```

```
draw_art()
```



Functions and Scope (Preview)

Question: Can a function access variables outside it?

```
name = "Agent Smith"

def greet():
    print(f"Hello, {name}") # ✅ Can read

greet() # Works!
```

Yes! But there are rules (we'll cover in Lesson 11).



Emily's Scope Teaser

"Variables have **scope** - where they can be accessed:

- **Global scope** - defined outside functions
- **Local scope** - defined inside functions

Functions can READ global variables, but writing to them requires special syntax (**global** keyword).

More on this next lesson!"

The `pass` Statement

What if you want to define a function but implement it later?

```
def future_feature():  
    pass # Placeholder - does nothing  
  
future_feature() # Runs without error
```

`pass` = "do nothing" statement. Useful for:

- Planning code structure
- Avoiding syntax errors during development

Challenge #3: Login System

Mission: Create a simple login interface using functions.

```
def print_banner():  
    # Print "=== LOGIN SYSTEM ==="  
  
def request_credentials():  
    # Print "Enter username:" and "Enter password:"  
  
def confirm_login():  
    # Print "✅ Login successful!"  
  
# Your main code here
```



Solution: Login System

```
def print_banner():  
    print("=" * 20)  
    print("  LOGIN SYSTEM")  
    print("=" * 20)  
  
def request_credentials():  
    print("Enter username:")  
    print("Enter password:")  
  
def confirm_login():  
    print("✅ Login successful!")  
  
# Main execution  
print_banner()  
request_credentials()  
confirm_login()
```



Cole's Professional Practice

"In real software development:

- Functions are typically **5-20 lines** long
- If a function is 50+ lines, consider splitting it
- Each function should do **one thing** well
- This is the **Single Responsibility Principle**

Good programmers spend time organizing code, not just writing it!"



Key Takeaways

- ✓ **Functions** are reusable blocks of code
- ✓ Use `def function_name():` to define
- ✓ Use `function_name()` to call
- ✓ **DRY principle** - Don't Repeat Yourself
- ✓ Functions create **stack frames** when called
- ✓ Proper **naming** and **organization** matter
- ✓ Functions can **call other functions**

Next Mission: Lesson 11

Coming Up:

- Function **parameters** - passing data in!
- **Return values** - getting data out!
- **Local vs global scope**
- Real-world function patterns

Agent Status: Function Basics MASTERED 



Mission Stats

Concept: Functions (Definition & Calling)

Difficulty: ★★ ★

Skills Unlocked:

- Code organization
- DRY principle
- Function calls
- Stack frame concept

Next Level: Parameters & Return Values

